```
    trigger=engineer,                  #4
)
task = """Write a snake game using Pygame."""

res = user_proxy.initiate_chat(
    recipient=engineer,
    message=task,
    max_turns=2,
    summary_method="last_msg"      #5
)
```

#1 This time, the assistant is given a system/persona message.
#2 A second assistant critic agent is created with a background.
#3 A custom function helps extract the code for review by the critic.
#4 A nested chat is created between the critic and the engineer.
#5 The proxy agent initiates a chat with a max delay and explicit summary method.

Run the `autogen_coding_critic.py` file in VS Code in debug mode, and watch the dialog between the agents. This time, after the code returns, the critic will also be triggered to respond. Then, the critic will add comments and suggestions to improve the code.

Nested chats work well for supporting and controlling agent interactions, but we'll see a better approach in the following section. Before that though, we'll review the importance of the AutoGen cache in the next section.

4.2.3 Understanding the AutoGen cache

AutoGen can consume many tokens over chat iterations as a conversable multi-agent platform. If you ask AutoGen to work through complex or novel problems, you may even encounter token limits on your LLM; because of this, AutoGen supports several methods to reduce token usage.

AutoGen uses caching to store progress and reduce token usage. Caching is enabled by default, and you may have already encountered it. If you check your current working folder, you'll notice a `.cache` folder, as shown in figure 4.8. Caching allows your agents to continue conversations if they get interrupted.